

Capítulo 3

Entorno, herramientas y configuración Full Stack

Desarrollo de Software IX · Código 1493 · Módulo II

Propósito del capítulo

Preparar al estudiante para instalar, configurar y verificar un entorno de desarrollo con Node.js, npm, Express, React, MongoDB, Postman, Visual Studio Code y una estructura de proyecto mantenible.

Tabla de contenido

1. Panorama del capítulo	3
2. El entorno como problema de ingeniería	3
3. Runtimes de JavaScript: Node.js, Bun y Deno	4
4. Gestión de dependencias: npm, package.json y semver	5
5. Configuración separada del código: variables de entorno	6
6. Control de versiones: Git como herramienta de colaboración	7
7. Editor y tooling: linters, formatters y consistencia	7
8. Contenedores y Docker: reproducibilidad del entorno completo	8
9. Configuración del stack del curso	9
10. Estructura del repositorio y scripts	11
11. Verificación del entorno	12
12. Errores frecuentes y su diagnóstico	13
13. Caso guía: preparar el entorno del sistema de solicitudes	13
14. Actividades sugeridas	14
15. Rúbrica breve	14
16. Cierre	14

1. Panorama del capítulo

La configuración del entorno no es una formalidad previa al “verdadero” trabajo de programar: es una decisión de ingeniería con consecuencias durante todo el ciclo de vida del proyecto. Un entorno mal preparado produce errores que parecen problemas de programación, retrasa al equipo y vuelve casi imposible reproducir el proyecto en otra máquina. Este capítulo se organiza en tres movimientos: primero estudia los conceptos transversales —qué es un runtime de JavaScript y por qué su modelo de ejecución importa, qué problema resuelven npm y los lockfiles, por qué la configuración debe vivir fuera del código, y qué papel cumplen Git, el tooling del editor y los contenedores—; después aplica esos conceptos a la configuración concreta del stack del curso (Node.js con Express, React con Vite, MongoDB); y cierra con un caso guía, actividades y una rúbrica breve.

Idea central

Un entorno Full Stack está listo cuando otro integrante puede clonar el repositorio, instalar dependencias con un solo comando, definir variables de entorno a partir de una plantilla, ejecutar frontend y backend, y probar un flujo conectado de extremo a extremo sin instrucciones improvisadas. La medida de calidad del entorno es la reproducibilidad, no la cantidad de herramientas instaladas.

2. El entorno como problema de ingeniería

Toda aplicación se ejecuta dentro de un contexto: un sistema operativo, una versión de runtime, un conjunto de bibliotecas, variables de configuración y servicios externos como la base de datos. A ese contexto se le llama entorno. El problema central es que el entorno de la máquina de un desarrollador casi nunca coincide exactamente con el de sus compañeros ni con el del servidor donde la aplicación correrá. Cuando esas diferencias no se gestionan, aparece el síntoma clásico: “en mi máquina funciona”. La frase no describe un misterio, sino una deuda: el conocimiento necesario para ejecutar el proyecto quedó implícito en una sola computadora en lugar de estar declarado en el repositorio.

La disciplina de configuración de entornos busca convertir ese conocimiento implícito en artefactos explícitos y versionados: un `package.json` que declara dependencias, un `lockfile` que congela versiones exactas, un archivo `.env.example` que enumera variables esperadas, scripts de npm que estandarizan comandos y, cuando el proyecto lo amerita, archivos de Docker que describen el entorno completo. Cada uno de estos artefactos responde a la misma pregunta desde un ángulo distinto: ¿qué necesita saber otra persona —u otra máquina— para ejecutar este sistema? Un entorno bien gestionado aporta cinco propiedades verificables:

- Reproducibilidad: cualquier integrante obtiene el mismo comportamiento siguiendo los mismos pasos documentados.
- Trazabilidad: las versiones de herramientas y bibliotecas quedan registradas y pueden auditarse cuando algo falla.
- Colaboración: el equipo comparte convenciones de formato, scripts y estructura, reduciendo fricción en cada integración.

- Seguridad: secretos y credenciales viven fuera del código fuente y nunca se publican en el repositorio.
- Velocidad: el tiempo de incorporación de un nuevo integrante se mide en minutos, no en días de configuración manual.

3. Runtimes de JavaScript: Node.js, Bun y Deno

JavaScript nació como lenguaje del navegador: durante años, su único entorno de ejecución fue el motor embebido en Chrome, Firefox o Safari. Un runtime es precisamente eso: el programa anfitrión que ejecuta código de un lenguaje y le ofrece servicios que el lenguaje por sí solo no tiene, como acceso al sistema de archivos, a la red o a procesos del sistema operativo. Node.js, creado en 2009 sobre el motor V8 de Chrome, sacó a JavaScript del navegador y le dio esos servicios, convirtiéndolo en un lenguaje apto para construir servidores. Esa decisión tuvo una consecuencia estratégica para el desarrollo Full Stack: un mismo lenguaje puede usarse en el frontend y en el backend, lo que reduce el costo cognitivo de cambiar de capa y permite compartir utilidades y formatos de datos entre ambas.

Lo distintivo de Node.js no es solo dónde corre JavaScript, sino cómo lo ejecuta. Un servidor pasa la mayor parte de su tiempo esperando: que la base de datos responda, que un archivo se lea, que un cliente termine de enviar datos. Los servidores tradicionales resolvían esa espera dedicando un hilo del sistema operativo a cada conexión; el hilo quedaba bloqueado mientras esperaba, y atender miles de conexiones exigía miles de hilos con su costo en memoria y cambios de contexto. Node.js adopta el modelo opuesto: un único hilo principal que nunca espera. Cada operación de entrada/salida se delega al sistema y se registra una función de continuación; el hilo sigue atendiendo otras tareas y, cuando la operación termina, el event loop —un ciclo que revisa permanentemente qué eventos han concluido— invoca la continuación correspondiente. Este modelo se llama E/S no bloqueante (non-blocking I/O) y explica tanto las fortalezas como las debilidades de Node.js: es excelente para aplicaciones dominadas por espera, como las APIs que consultan bases de datos, y es débil para cálculo intensivo, porque un cálculo pesado en el hilo principal congela a todos los clientes a la vez. En el código, este modelo se expresa mediante promesas y la sintaxis `async/await`, que permiten escribir lógica asíncrona con apariencia secuencial.

Concepto clave

El event loop es la razón por la que Node.js maneja miles de conexiones con un solo hilo: las operaciones lentas no detienen al programa, sino que registran una continuación que se ejecuta cuando el resultado está disponible. El precio es la regla que el estudiante verá repetida en este curso: nunca bloquear el event loop con tareas pesadas.

Node.js ya no es el único runtime de JavaScript del lado servidor. Deno (2018), creado por el propio autor original de Node.js, repensó decisiones históricas: incorpora seguridad por permisos explícitos, soporte nativo de TypeScript y herramientas integradas. Bun (2022) prioriza el rendimiento —usa el motor JavaScriptCore en lugar de V8— e integra en un solo binario el runtime, el gestor de paquetes, el empaquetador y el ejecutor de pruebas. La existencia de alternativas no obliga a cambiar de herramienta, pero enseña una lección de criterio: un runtime es una decisión técnica con ventajas y compromisos, no un hecho natural. En este curso se trabaja con Node.js por su madurez, la amplitud de su ecosistema y su presencia dominante en la industria, sabiendo que los conceptos —event loop, módulos, gestión de paquetes— se transfieren a cualquier runtime moderno.

Runtime	Año	Rasgo distintivo	Cuándo considerarlo
Node.js	2009	Madurez, ecosistema npm masivo, motor V8.	Opción por defecto en proyectos profesionales y en este curso.
Deno	2018	Seguridad por permisos, TypeScript nativo, tooling integrado.	Proyectos que priorizan seguridad y simplicidad de herramientas.
Bun	2022	Rendimiento, binario todo-en-uno (runtime, paquetes, pruebas).	Prototipos rápidos y servicios donde la velocidad de arranque importa.

4. Gestión de dependencias: npm, package.json y semver

Ninguna aplicación moderna se escribe desde cero: se construye sobre bibliotecas de terceros —Express para HTTP, Mongoose para MongoDB, React para interfaces— que a su vez dependen de otras bibliotecas, formando un árbol de cientos o miles de paquetes. Gestionar ese árbol a mano es inviable: habría que descargar cada paquete, resolver sus dependencias transitivas, detectar conflictos de versiones y repetir el proceso en cada máquina. npm (Node Package Manager) automatiza todo eso: resuelve el árbol completo, descarga los paquetes a la carpeta `node_modules` y registra lo decidido en archivos que el equipo comparte. La carpeta `node_modules` es un artefacto derivado —pesado y regenerable— y por eso nunca se versiona en Git: se reconstruye con `npm install` a partir de los archivos que sí se versionan.

El archivo `package.json` es el manifiesto del proyecto: declara su nombre, sus scripts de ejecución y, sobre todo, sus dependencias con rangos de versión. Distingue entre dependencias —paquetes que la aplicación necesita para funcionar en producción, como Express— y `devDependencies` —herramientas que solo se usan durante el desarrollo, como nodemon o un linter—. Esa distinción no es cosmética: en un despliegue de producción pueden omitirse las dependencias de desarrollo, reduciendo el tamaño de instalación y la superficie de ataque.

```
{
  "name": "server",
  "scripts": {
    "dev": "nodemon src/index.js",
    "start": "node src/index.js",
    "test": "node --test"
  },
  "dependencies": {
    "express": "^4.19.2",
    "mongoose": "^8.4.0"
  },
  "devDependencies": {
    "nodemon": "^3.1.0"
  }
}
```

Los rangos como `^4.19.2` siguen la convención de versionado semántico (semver): una versión MAJOR.MINOR.PATCH donde el primer número cambia cuando hay rupturas de compatibilidad, el segundo cuando se agregan funciones compatibles y el tercero cuando solo se corrigen errores. El prefijo `^` autoriza a npm a instalar versiones menores y parches más recientes, pero nunca una versión mayor. El sistema funciona como un contrato social: el autor del paquete promete no romper compatibilidad sin subir el número

mayor, y el consumidor decide cuánto riesgo acepta con el rango que declara. Ahora bien, un rango admite múltiples versiones concretas, y dos instalaciones hechas en fechas distintas podrían resolver árboles diferentes. Para eliminar esa ambigüedad existe el lockfile (package-lock.json): un registro exacto de cada paquete instalado, con su versión precisa y su huella criptográfica de integridad. El lockfile se versiona en Git y garantiza que todo el equipo —y el servidor de despliegue— instale exactamente el mismo árbol con npm ci. La pareja package.json + lockfile resume la filosofía del capítulo: el primero expresa la intención (qué rangos acepto), el segundo congela la realidad (qué quedó instalado).

Símbolo	Significado	Ejemplo con 4.19.2
4.19.2	Versión exacta, sin flexibilidad.	Solo 4.19.2.
^4.19.2	Acepta minor y patch dentro de la misma mayor.	4.19.3, 4.20.0; nunca 5.0.0.
~4.19.2	Acepta solo parches dentro de la misma menor.	4.19.5; nunca 4.20.0.
*	Cualquier versión (práctica desaconsejada).	Comportamiento impredecible.

5. Configuración separada del código: variables de entorno

Una misma aplicación se ejecuta en contextos distintos: la laptop de cada integrante, un servidor de pruebas, producción. Lo que cambia entre contextos no es el código sino la configuración: el puerto de escucha, la cadena de conexión a la base de datos, el secreto con el que se firman los tokens, la URL del frontend autorizada por CORS. La metodología conocida como los doce factores (12-factor app), un conjunto de principios formulado a partir de la experiencia operando software en la nube, dedica su tercer factor exactamente a esto: la configuración que varía entre despliegues debe almacenarse en el entorno, no en el código. La prueba ácida es simple: ¿podría publicarse el código fuente ahora mismo sin exponer ninguna credencial? En Node.js esa separación se materializa con variables de entorno leídas desde process.env, normalmente cargadas en desarrollo desde un archivo .env que está excluido de Git mediante .gitignore. Para que el resto del equipo sepa qué variables se esperan, se versiona una plantilla .env.example con los nombres reales y valores ficticios. Hay además una práctica de robustez que distingue un proyecto cuidado: validar la configuración al arrancar; si falta una variable crítica, la aplicación debe fallar de inmediato con un mensaje claro, no horas después con un error críptico.

Variable	Uso	Ejemplo seguro para .env.example
PORT	Puerto donde escucha el backend.	3001
MONGODB_URI	Cadena de conexión a MongoDB.	mongodb://localhost:27017/app
JWT_SECRET	Secreto para firmar tokens de sesión.	cambiar-por-valor-local
CLIENT_URL	Origen del frontend permitido por CORS.	http://localhost:5173
NODE_ENV	Modo de ejecución (development/production).	development

```
# server/.env.example (se versiona; sin secretos reales)
PORT=3001
MONGODB_URI=mongodb://localhost:27017/app
JWT_SECRET=cambiar-por-valor-local
CLIENT_URL=http://localhost:5173
```

```
# server/.gitignore
node_modules/
.env
```

Regla de seguridad

Nunca publicar .env con claves reales. Si un secreto llega a subirse al repositorio, borrarlo del archivo no basta: queda en el historial de Git. El secreto debe rotarse (generarse de nuevo) y el archivo excluirse en .gitignore de ahí en adelante.

6. Control de versiones: Git como herramienta de colaboración

Git suele presentarse como un mecanismo para “guardar versiones”, pero su valor en un equipo Full Stack es más profundo: es la infraestructura sobre la que se coordina el trabajo paralelo. Cada commit es una fotografía coherente del proyecto acompañada de una explicación; cada rama es una línea de trabajo aislada que puede avanzar sin romper lo que ya funciona; cada integración (merge o pull request) es un punto de revisión donde el equipo discute el cambio antes de incorporarlo. Visto así, el historial de Git es también documentación: responde por qué el código llegó a ser como es, algo que ningún comentario en el código responde con la misma fidelidad. Para que esa coordinación funcione, el equipo necesita convenciones explícitas, y la configuración del entorno y el control de versiones se refuerzan mutuamente: el repositorio solo es reproducible si contiene los artefactos correctos y excluye los incorrectos.

- Confirmar cambios pequeños y coherentes: un commit, una intención verificable.
- Escribir mensajes que expliquen el porqué, no solo el qué (“corrige validación de correo vacío”, no “cambios”).
- Mantener la rama principal siempre ejecutable; el trabajo en progreso vive en ramas cortas por funcionalidad.
- Integrar con frecuencia para reducir conflictos: un conflicto pequeño hoy es mejor que uno grande la próxima semana.
- Excluir de Git todo lo regenerable (node_modules, dist) y todo lo secreto (.env).

7. Editor y tooling: linters, formatters y consistencia

Visual Studio Code es el editor de referencia del curso, pero la decisión relevante no es la marca del editor sino el tooling que lo acompaña. Los linters (como ESLint) analizan el código sin ejecutarlo y detectan patrones propensos a error: variables sin usar, comparaciones sospechosas, promesas sin manejar; funcionan como una revisión automática y constante que atrapa defectos antes de que lleguen siquiera a un commit. Los formatters (como Prettier) normalizan la presentación del código —sangrías, comillas, longitud

de línea— aplicando reglas mecánicas idénticas para todo el equipo. El argumento a favor de la consistencia no es estético sino económico y cognitivo: cuando todo el código tiene el mismo formato, los diffs de Git muestran únicamente cambios de lógica y no ruido de estilo, las revisiones se concentran en lo importante y nadie gasta tiempo discutiendo dónde van las llaves. La regla práctica es delegar al formatter todo lo mecánico y reservar el juicio humano para lo que sí lo requiere: nombres, diseño y corrección. Estas herramientas se declaran como devDependencies y se configuran con archivos versionados, de modo que la convención viaja con el proyecto y no depende de la configuración personal de cada máquina.

Herramienta	Qué hace	Problema que evita
ESLint	Analiza el código estáticamente contra reglas configurables.	Errores latentes: variables sin usar, promesas ignoradas, código inalcanzable.
Prettier	Reescribe el código con formato uniforme y automático.	Discusiones de estilo y diffs contaminados por cambios cosméticos.
EditorConfig	Unifica sangrías y fin de línea entre editores distintos.	Inconsistencias invisibles entre sistemas operativos y editores.
Extensiones de VS Code	Integran linter, formatter y depurador en el flujo de edición.	Retroalimentación tardía: el error se ve al editar, no al ejecutar.

8. Contenedores y Docker: reproducibilidad del entorno completo

Los artefactos vistos hasta aquí —lockfiles, .env.example, scripts— declaran las dependencias de la aplicación, pero no el entorno donde corre: la versión exacta de Node.js, el sistema operativo, la instancia de MongoDB. Los contenedores atacan ese nivel. Un contenedor es un proceso aislado que se ejecuta con su propio sistema de archivos, sus propias bibliotecas y su propia configuración de red, compartiendo únicamente el núcleo del sistema operativo anfitrión. A diferencia de una máquina virtual, no emula hardware ni arranca un sistema operativo completo, por lo que inicia en segundos y consume una fracción de los recursos. Docker es la plataforma que popularizó este modelo al empaquetarlo en herramientas accesibles.

La distinción conceptual fundamental separa imagen de contenedor. Una imagen es una plantilla inmutable —el sistema de archivos completo más metadatos— construida a partir de una receta llamada Dockerfile, que describe capa por capa cómo se prepara el entorno: de qué base se parte, qué archivos se copian, qué dependencias se instalan, qué comando arranca la aplicación. Un contenedor es una instancia en ejecución de esa imagen: de una misma imagen pueden lanzarse muchos contenedores idénticos. La relación es análoga a la de clase e instancia en programación orientada a objetos. La consecuencia práctica es poderosa: si la imagen se construye una vez y se ejecuta igual en cualquier máquina con Docker, la frase “en mi máquina funciona” pierde sentido, porque la máquina relevante viaja con el proyecto.

Una aplicación Full Stack no es un solo proceso: como mínimo conviven la API y la base de datos. docker-compose permite declarar ese conjunto en un archivo YAML versionado: qué servicios existen, de qué imagen parte cada uno, qué puertos exponen, qué variables de entorno reciben y qué volúmenes persisten sus datos. Con un único comando, cualquier integrante levanta el sistema completo —incluyendo un MongoDB correctamente configurado— sin instalar la base de datos en su máquina. Para un equipo académico, este es el uso más rentable de Docker: eliminar la instalación manual de servicios de

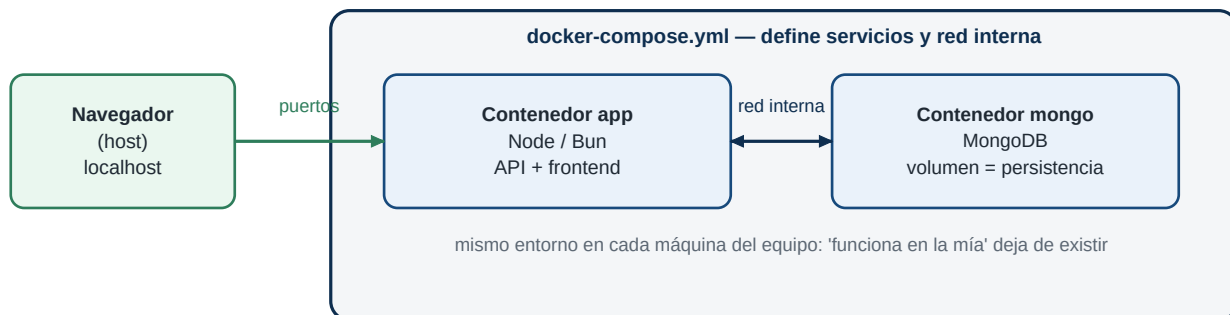
infraestructura.

```
# docker-compose.yml (fragmento ilustrativo)
services:
  mongo:
    image: mongo:7
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db
  api:
    build: ./server
    ports:
      - "3001:3001"
    environment:
      - MONGODB_URI=mongodb://mongo:27017/app
    depends_on:
      - mongo
volumes:
  mongo-data:

# Levantar todo el sistema:
# docker compose up
```

Perspectiva

En este curso, Docker es opcional pero conceptualmente obligatorio: aun si el estudiante instala MongoDB de forma nativa, debe comprender que los contenedores son la respuesta de la industria al problema de la reproducibilidad de entornos, y que un docker-compose.yml es documentación ejecutable de la infraestructura del proyecto.



9. Configuración del stack del curso

Las secciones anteriores dieron el marco teórico; esta lo aplica al stack concreto del curso. Cada herramienta ocupa una posición deliberada en la arquitectura: Node.js y Express forman la capa de servicios, React con Vite la capa de presentación, MongoDB la capa de persistencia, y Postman es el instrumento para probar el contrato HTTP de forma aislada, antes de que el frontend exista: una colección de Postman compartida documenta qué rutas hay, qué esperan y qué devuelven, y permite saber de qué lado está un problema de integración. La tabla siguiente resume el papel de cada pieza y la evidencia observable de que quedó bien instalada.

Herramienta	Uso en el curso	Evidencia de instalación
Visual Studio Code	Editar código, terminal integrada, extensiones de linting y formato.	Proyecto abierto, terminal funcional y formato consistente al guardar.
Node.js	Ejecutar JavaScript del lado servidor y el tooling del frontend.	node -v responde con una versión LTS compatible.
npm	Gestionar dependencias y scripts del proyecto.	npm -v responde; package.json y lockfile generados.
Express.js	Construir la API HTTP del backend.	El servidor responde en el puerto local configurado.
React + Vite	Construir la interfaz por componentes con recarga instantánea.	La aplicación se sirve en el navegador en modo desarrollo.
MongoDB	Persistir los datos del proyecto.	Conexión local (o contenedor, o Atlas) aceptando consultas.
Postman	Probar endpoints antes y después de conectar el frontend.	Colección con las rutas principales probadas y guardadas.

9.1 Backend: Node.js, npm y Express

La configuración del backend comienza eligiendo una versión LTS (Long Term Support) de Node.js: las versiones LTS reciben correcciones durante años y son las que la industria despliega en producción. Sobre esa base, npm init crea el manifiesto del proyecto y npm install incorpora Express como dependencia de producción, junto con dotenv (carga el .env hacia process.env) y cors (autoriza al frontend, servido desde otro origen durante el desarrollo, a consumir la API); nodemon —que reinicia el servidor al detectar cambios— entra como dependencia de desarrollo. La decisión estructural importante es leer toda la configuración desde process.env, de modo que el mismo código sirva para desarrollo y producción sin modificarse, y crear desde el primer día una ruta de salud (por ejemplo GET /health) que responda con un estado simple: es la sonda con la que se verifica que el servidor vive, antes de que exista cualquier lógica de negocio.

```
# Crear y configurar el proyecto backend
mkdir server && cd server
npm init -y
npm install express mongoose dotenv cors
npm install --save-dev nodemon

# Verificación mínima del entorno
node -v      # versión LTS instalada
npm -v      # gestor de paquetes disponible
npm run dev  # servidor con recarga automática
```

9.2 Frontend: React con Vite

React es una biblioteca, no un framework completo: no dicta cómo se sirve, se empaqueta ni se transforma el código, y por eso necesita una herramienta de construcción. La elección del curso es Vite, y la razón es conceptual además de práctica: los empaquetadores tradicionales procesaban toda la aplicación antes de mostrar la primera pantalla, con tiempos de arranque que crecían con el proyecto; Vite sirve los módulos bajo demanda usando los módulos nativos del navegador (ESM) durante el desarrollo, logrando arranque

casi instantáneo y recarga en caliente que conserva el estado de la aplicación, y para producción sí empaqueta y optimiza. La decisión de configuración relevante en el frontend es centralizar el acceso a la API en un módulo de servicios: un único archivo que conoce la URL base del backend (leída de una variable de entorno de Vite, con prefijo `VITE_`) y expone funciones que los componentes consumen sin conocer URLs. Esa indirección, que parece burocracia en un proyecto pequeño, es la que permite cambiar el backend de puerto o de servidor sin tocar ningún componente.

```
# Crear el proyecto frontend con Vite
npm create vite@latest client -- --template react
cd client
npm install
npm run dev      # sirve la aplicación en http://localhost:5173

# client/.env.example
VITE_API_URL=http://localhost:3001/api
```

9.3 Base de datos: MongoDB

MongoDB es una base de datos orientada a documentos: almacena estructuras JSON (técnicamente BSON) agrupadas en colecciones, sin exigir un esquema rígido previo. Esa flexibilidad encaja con un stack JavaScript —el mismo formato de datos viaja del navegador a la API y de la API a la base— y con proyectos cuyo modelo de datos evoluciona durante el desarrollo. La flexibilidad, sin embargo, no exime de disciplina: en el curso el esquema se declara en la capa de aplicación mediante Mongoose, que define la forma y las validaciones de cada documento. Para disponer de una instancia hay tres caminos: instalación nativa, contenedor Docker (la opción más limpia, como se vio en la sección 8) o el servicio gestionado MongoDB Atlas, que ofrece un clúster gratuito accesible por una cadena de conexión. Cualquiera de los tres es válido siempre que la URI quede en el `.env` y nunca en el código.

```
# Opción A: MongoDB en contenedor (recomendada)
docker run -d --name mongo -p 27017:27017 \
  -v mongo-data:/data/db mongo:7

# Opción B: verificación de una instancia local
mongosh --eval "db.runCommand({ ping: 1 })"

# La aplicación siempre se conecta vía variable de entorno:
# MONGODB_URI=mongodb://localhost:27017/app
```

10. Estructura del repositorio y scripts

La estructura del proyecto debe facilitar la navegación y reflejar la separación de responsabilidades estudiada en el capítulo anterior. Para proyectos del tamaño de este curso conviene un monorepo: un único repositorio con las carpetas `client` y `server`, cada una con su propio `package.json`, más una carpeta `docs` y un `README` raíz que explica cómo ejecutar el sistema completo. La alternativa —dos repositorios separados— se justifica en equipos grandes con ciclos de despliegue independientes, pero a esta escala duplica la gestión sin beneficio. Los scripts de npm completan la reproducibilidad: en lugar de pedir a cada integrante que memorice comandos largos, el `package.json` de cada parte declara verbos estándar; quien sabe que `npm run dev` arranca el modo desarrollo en cualquier repositorio bien organizado no necesita instrucciones especiales en cada uno.

Ruta	Propósito	Notas
/client	Aplicación React.	Componentes, rutas, servicios HTTP, estilos y assets.
/server	API Express.	Rutas, controladores, modelos, middlewares y configuración.
/docs	Documentación técnica.	Diagramas, endpoints, decisiones y capturas.
/server/.env.example	Plantilla de variables de entorno.	Nombres reales, valores ficticios; nunca secretos.
/docker-compose.yml	Infraestructura declarada (si se usa Docker).	Levanta API y MongoDB con un comando.
/README.md	Entrada principal del proyecto.	Instalación, ejecución, endpoints y usuarios de prueba.

Script	Dónde	Función
npm run dev	client	Ejecuta React con Vite en modo desarrollo.
npm run dev	server	Ejecuta Express con recarga automática (nodemon).
npm start	server	Ejecuta el backend en modo normal (producción).
npm test	client / server	Corre las pruebas configuradas.
npm run seed	server	Carga datos semilla para desarrollo y demostraciones.

11. Verificación del entorno

La verificación convierte la instalación en evidencia. No basta con que cada herramienta exista: hay que comprobar que trabajan juntas, recorriendo el camino completo desde el clonado hasta un flujo conectado. La secuencia siguiente es, en la práctica, la misma que vivirá cualquier integrante nuevo del equipo; ejecutarla deliberadamente revela los pasos que faltan en la documentación.

- 1 Clonar el repositorio en una ruta limpia (idealmente en una máquina o usuario distinto al del autor).
- 2 Instalar dependencias de backend y frontend con `npm install` en cada carpeta, verificando que el lockfile se respete.
- 3 Crear el archivo `.env` a partir de `.env.example` y completar los valores locales.
- 4 Levantar MongoDB (servicio local, contenedor o Atlas) y comprobar la conexión.
- 5 Ejecutar el backend y probar la ruta `/health` o equivalente desde el navegador o Postman.
- 6 Ejecutar el frontend y confirmar que consume al menos un endpoint real del backend.
- 7 Probar los endpoints principales en Postman y guardar la colección.
- 8 Registrar en el README cualquier paso que haya requerido conocimiento no documentado.

12. Errores frecuentes y su diagnóstico

Síntoma	Causa probable	Corrección
El backend no inicia.	Puerto ocupado o variable de entorno faltante.	Cambiar PORT o validar el .env contra .env.example.
React no conecta con la API.	URL base incorrecta o CORS bloqueando el origen.	Revisar el módulo de servicios HTTP y la configuración de CORS.
MongoDB falla al conectar.	URI incorrecta, servicio apagado o credenciales inválidas.	Probar la conexión con mongosh y revisar la URI del .env.
Funciona en una máquina y en otra no.	Versiones de Node o dependencias no congeladas.	Usar la misma versión LTS, instalar con npm ci, actualizar README.
Credenciales expuestas en el repositorio.	Se versionó el .env con secretos reales.	Retirar el archivo, rotar el secreto y reforzar .gitignore.
Una dependencia se comporta distinto entre integrantes.	Lockfiles divergentes o instalación sin lockfile.	Unificar package-lock.json y reinstalar desde cero.

13. Caso guía: preparar el entorno del sistema de solicitudes

Retomemos el caso del capítulo anterior: una aplicación para gestionar solicitudes académicas, donde el estudiante crea una solicitud y un administrador cambia su estado. Antes de escribir la primera ruta, el equipo debe tomar las decisiones de entorno de este capítulo y dejarlas materializadas en el repositorio. El ejercicio interesante no es ejecutar comandos, sino justificar cada decisión con el riesgo que evita.

Decisión de entorno	Aplicación en el caso	Riesgo si se ignora
Monorepo client/server con scripts npm.	Un solo repositorio; npm run dev en cada carpeta arranca cada capa.	Instrucciones de ejecución dispersas y dependientes de la memoria de un integrante.
Versión LTS de Node fijada y documentada.	Todo el equipo desarrolla sobre la misma versión declarada en el README.	Errores fantasma por diferencias de runtime entre máquinas.
Lockfile versionado e instalación con npm ci.	El árbol de dependencias es idéntico para todos.	"A mí me funciona" con versiones distintas de Express o Mongoose.
.env.example con PORT, MONGODB_URI, JWT_SECRET y CLIENT_URL.	Cada integrante crea su .env local en un minuto.	Secretos en el código o configuración indiscrutable para los demás.
MongoDB en contenedor con docker-compose.	docker compose up levanta base de datos y API juntas.	Horas perdidas instalando y configurando MongoDB en cada máquina.
Colección Postman con las rutas de solicitudes.	El contrato HTTP se prueba antes de construir las pantallas.	Errores de integración indistinguibles entre frontend y backend.

14. Actividades sugeridas

- Crear un repositorio con carpetas client, server y docs, con package.json y script dev en cada parte.
- Configurar un endpoint /health en Express y consumirlo desde un componente React.
- Crear .env.example, cargar las variables con dotenv y hacer que el servidor falle con mensaje claro si falta una crítica.
- Explicar por escrito, en media página, qué garantiza el package-lock.json que el package.json no garantiza.
- Levantar MongoDB con Docker (o documentar la alternativa elegida) y conectar Mongoose usando la URI del entorno.
- Preparar una colección de Postman con al menos cuatro endpoints y compartirla con el equipo.
- Pedir a un compañero que clone el repositorio y lo ejecute siguiendo solo el README; registrar y corregir cada paso que falló.

15. Rúbrica breve

Criterio	Excelente	Satisfactorio	Insuficiente
Reproducibilidad	Otro integrante ejecuta el sistema completo siguiendo solo el README.	El sistema corre con ayuda puntual del autor.	Solo corre en la máquina original.
Gestión de dependencias	Lockfile versionado, node_modules excluido, semver comprendido y aplicado.	Dependencias instaladas pero sin control deliberado de versiones.	node_modules versionado o versiones inconsistentes.
Configuración	Variables en .env, plantilla .env.example completa, validación al arrancar.	Variables usadas pero plantilla incompleta.	Configuración o secretos incrustados en el código.
Comprensión teórica	Explica event loop, semver y contenedores con sus porqués.	Define los conceptos sin conectarlos con decisiones.	Repite comandos sin explicar qué problema resuelven.

16. Cierre

Un entorno bien configurado permite que el equipo se concentre en diseñar e implementar, en lugar de combatir problemas accidentales. Pero la lección de este capítulo va más allá de la comodidad: cada herramienta estudiada —el runtime y su event loop, npm y sus lockfiles, las variables de entorno, Git, el tooling del editor, los contenedores— es la respuesta de la ingeniería de software a una pregunta concreta de reproducibilidad, trazabilidad o colaboración. Quien entiende esas preguntas puede evaluar herramientas nuevas con criterio propio, porque las herramientas cambian pero los problemas permanecen. En los capítulos siguientes, este entorno deja de ser el tema y se convierte en el suelo firme sobre el que se construyen el backend, la persistencia y la interfaz del proyecto.